

LAPORAN PRAKTIKUM 11

Nama : Akmallutfi Afza Gusty

NIM : 707012500064

Kelas : D4 SIKC 49-04

Mata kuliah : Pemrograman Web

BAB I - TUJUAN PRAKTIKUM

Praktikum ini bertujuan agar mahasiswa mampu:

- a. Memahami konsep dasar React.js sebagai library JavaScript untuk membangun antarmuka pengguna (UI).
- b. Menerapkan konsep component dalam React, baik parent component maupun child component.
- c. Menggunakan props untuk mengirimkan data dari parent component ke child component.
- d. Mengelola state aplikasi menggunakan React Hook useState.
- e. Menerapkan rendering list menggunakan fungsi map() untuk menampilkan data array.
- f. Mengimplementasikan conditional rendering untuk menampilkan konten berbeda berdasarkan kondisi.
- g. Membangun fitur tambah dan hapus data secara dinamis tanpa backend.
- h. Membuat project React menggunakan Vite sebagai build tool modern.

BAB II - ALAT DAN BAHAN

3.1 Perangkat Keras

- Laptop/Komputer dengan RAM minimal 4 GB
- Koneksi internet (untuk instalasi dependencies)

3.2 Perangkat Lunak

Perangkat Lunak	Versi	Keterangan
Node.js	v24.14.0	Runtime JavaScript
npm	v11.9.0	Package manager
Vite	v8.0.13	Build tool & dev server
React	v19.x	Library UI JavaScript
Visual Studio Code	Terbaru	Code editor
Browser Modern	Chrome/Firefox/Edge	Untuk menjalankan aplikasi

BAB III - LANGKAH-LANGKAH PRAKTIKUM

Langkah 1: Membuat Project React dengan Vite

Buka terminal/command prompt, lalu jalankan perintah berikut untuk membuat project baru:

```
cmd /c "npm create-vite@latest aktivitas-mahasiswa --template react"

cd aktivitas-mahasiswa

npm install
```

Perintah di atas akan membuat folder project bernama aktivitas-mahasiswa dengan template React, kemudian menginstal semua dependencies yang dibutuhkan.

Langkah 2: Menyiapkan Struktur Folder

Buat folder components di dalam folder src:

```
src/

App.jsx          <- Parent Component

App.css          <- Styling

main.jsx         <- Entry Point

index.css        <- Global Reset

components/

  ActivityItem.jsx <- Child Component

  ActivityList.jsx <- Child Component
```

Struktur ini memisahkan komponen-komponen React agar lebih terorganisir dan mudah dipelihara.

Langkah 3: Membuat Child Component ActivityItem.jsx

Buat file src/components/ActivityItem.jsx yang bertugas menampilkan satu item aktivitas beserta tombol hapus:

```
function ActivityItem({ nomor, aktivitas, onHapus }) {

  return (
```

```

<li className="activity-item">

  <div className="activity-info">

    <span className="activity-number">{nomor}</span>

    <span className="activity-name">{aktivitas}</span>

  </div>

  <button className="btn-hapus" onClick={onHapus}>

    Hapus

  </button>

</li>

);
}

export default ActivityItem;

```

Komponen ini menerima tiga props: nomor (urutan), aktivitas (nama kegiatan), dan onHapus (fungsi callback dari parent).

Langkah 4: Membuat Child Component ActivityList.jsx

Buat file src/components/ActivityList.jsx yang menerapkan conditional rendering dan rendering list:

```

import ActivityItem from '../ActivityItem';

function ActivityList({ daftarAktivitas, onHapus }) {

  // Conditional rendering: jika kosong

  if (daftarAktivitas.length === 0) {

    return <p>Belum ada aktivitas</p>;

  }

  // Rendering list dengan map()

  return (

    <ul>

      {daftarAktivitas.map((item, index) => (

        <ActivityItem

```

```

        key={item.id}

        nomor={index + 1}

        aktivitas={item.nama}

        onDelete={ () => onDelete (item.id) }

      />

    ) ) }

  </ul>

);

}

export default ActivityList;

```

Komponen ini menggunakan if statement untuk conditional rendering. Jika array kosong ditampilkan pesan, jika tidak kosong ditampilkan list menggunakan map().

Langkah 5: Membuat Parent Component App.jsx

Edit file src/App.jsx sebagai komponen utama yang mengelola semua state:

```

import { useState } from 'react';

import ActivityList from './components/ActivityList';

function App() {

  const [daftarAktivitas, setDaftarAktivitas] = useState([

    { id: 1, nama: 'Mengerjakan tugas pemrograman web' },

    { id: 2, nama: 'Belajar konsep React.js' },

  ]);

  const [inputAktivitas, setInputAktivitas] = useState('');

  const [nextId, setNextId] = useState(3);

  const tambahAktivitas = () => {

    if (inputAktivitas.trim() === '') return;

    setDaftarAktivitas([...daftarAktivitas,

```

```

        { id: nextId, nama: inputAktivitas.trim() }]);

    setInputAktivitas('');

    setNextId(nextId + 1);

  };

const hapusAktivitas = (id) => {

  setDaftarAktivitas(daftarAktivitas.filter(item => item.id !== id));

};

return (

  <div>

    <input value={inputAktivitas}

      onChange={e => setInputAktivitas(e.target.value)} />

    <button onClick={tambahAktivitas}>Tambah</button>

    <ActivityList daftarAktivitas={daftarAktivitas}

      onHapus={hapusAktivitas} />

  </div>

);

}

export default App;

```

App.jsx adalah parent component yang menyimpan state utama. Data dikirim ke ActivityList melalui props daftarAktivitas dan onHapus.

Langkah 6: Menjalankan Aplikasi

Jalankan development server dengan perintah:

```
npm run dev
```

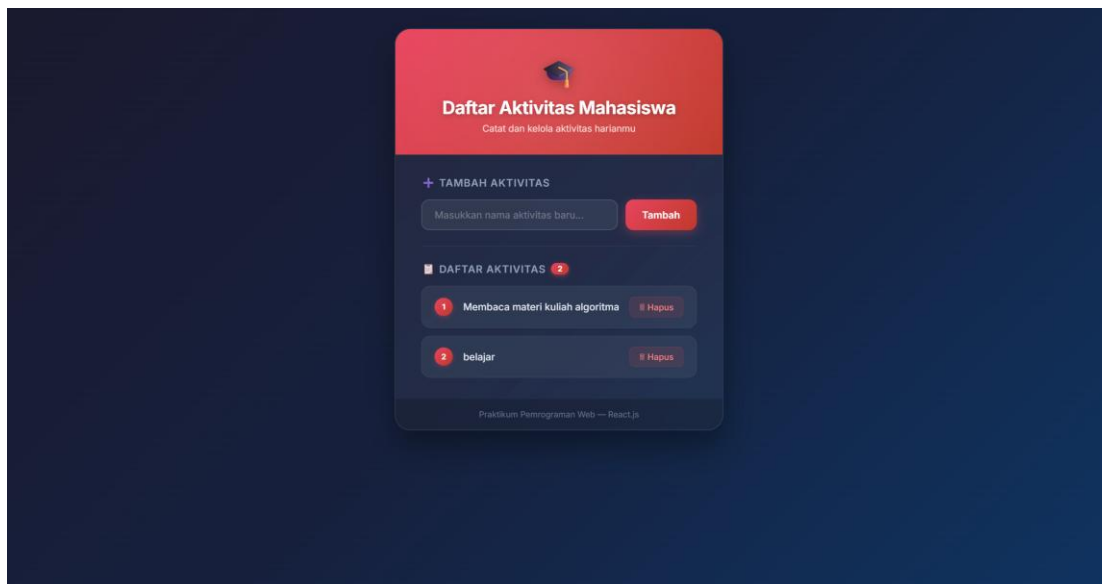
Buka browser dan akses <http://localhost:5173/> untuk melihat aplikasi yang berjalan.

BAB IV - HASIL DAN DOKUMENTASI PROGRAM

Setelah seluruh langkah praktikum dilaksanakan, aplikasi berhasil dijalankan pada browser dengan alamat <http://localhost:5173/>. Berikut adalah hasil dan dokumentasi program:

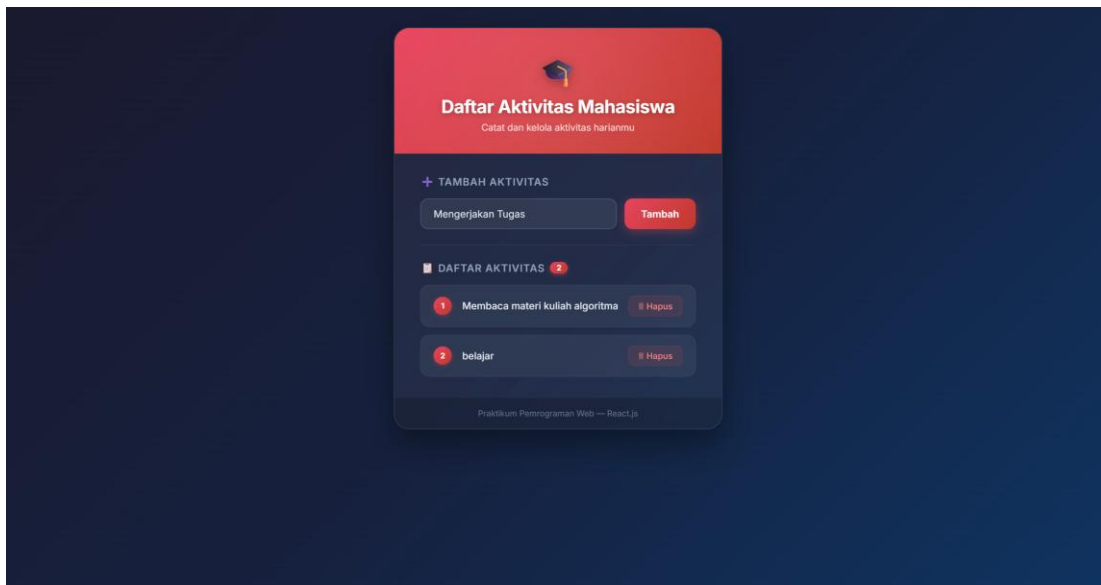
5.1 Tampilan Awal Aplikasi

Aplikasi menampilkan header dengan judul 'Daftar Aktivitas Mahasiswa', form input untuk menambah aktivitas baru, dan daftar 3 aktivitas default yang tersimpan dalam state awal. Setiap item menampilkan nomor urut, nama aktivitas, dan tombol Hapus.



5.2 Fitur Tambah Aktivitas

Pengguna dapat mengetik nama aktivitas pada field input lalu menekan tombol 'Tambah' atau menekan Enter. Aktivitas baru langsung muncul di bagian bawah daftar dengan nomor urut yang otomatis bertambah. Input juga divalidasi — jika field kosong, aktivitas tidak akan ditambahkan.



BAB V - ANALISIS DAN PEMBAHASAN

6.1 Component dan Hierarki

Aplikasi ini dibangun menggunakan tiga komponen React yang terstruktur secara hierarkis. `App.jsx` bertindak sebagai parent component dan pusat pengelolaan state. `ActivityList.jsx` adalah child component pertama yang menerima daftar aktivitas dan menentukan tampilan (kosong atau berisi). `ActivityItem.jsx` adalah child component terdalam yang menampilkan satu aktivitas beserta tombol hapus. Pola hierarki ini sesuai dengan prinsip component-based architecture di React, di mana setiap komponen memiliki tanggung jawab yang jelas dan terfokus.

6.2 State Management dengan `useState`

Aplikasi menggunakan tiga state yang dikelola dengan React Hook `useState`: (1) `daftarAktivitas` — array of objects yang menjadi sumber data utama, (2) `inputAktivitas` — string yang menyimpan nilai input field secara real-time (controlled component), dan (3) `nextId` — integer auto-increment sebagai identifier unik setiap aktivitas. Prinsip immutability diterapkan dengan benar: state tidak pernah dimutasi langsung, melainkan selalu menghasilkan array baru menggunakan spread operator (...) untuk tambah dan `filter()` untuk hapus.

6.3 Props dan Alur Data Satu Arah

React menerapkan konsep unidirectional data flow (alur data satu arah). Data mengalir dari parent ke child melalui props, tidak pernah sebaliknya. Ketika child perlu memodifikasi state parent (seperti menghapus aktivitas), child tidak mengakses state langsung melainkan memanggil fungsi callback yang dikirim parent sebagai props (onHapus). Pola ini membuat alur data lebih mudah diprediksi dan di-debug.

6.4 Rendering List dengan map()

Daftar aktivitas ditampilkan menggunakan fungsi `map()` yang mengiterasi setiap elemen array dan mengembalikan JSX element. Setiap elemen wajib memiliki prop `key` yang unik — dalam aplikasi ini menggunakan `item.id` — agar React dapat mengidentifikasi perubahan elemen secara efisien melalui Virtual DOM reconciliation.

6.5 Conditional Rendering

`ActivityList.jsx` mengimplementasikan conditional rendering menggunakan `if` statement biasa. Ketika `daftarAktivitas.length === 0`, komponen mengembalikan JSX empty state. Jika tidak kosong, komponen mengembalikan JSX daftar. Di `App.jsx` juga terdapat conditional rendering inline menggunakan `&&` operator untuk menampilkan badge jumlah hanya ketika daftar tidak kosong. Kedua teknik ini adalah cara umum conditional rendering di React.

BAB VI - KESIMPULAN

Berdasarkan praktikum yang telah dilaksanakan, dapat disimpulkan beberapa hal sebagai berikut:

- a. React.js adalah library JavaScript yang powerful untuk membangun antarmuka pengguna berbasis komponen yang dinamis dan reaktif terhadap perubahan data.
- b. Konsep component memungkinkan pembagian UI menjadi bagian-bagian kecil yang reusable dan mudah dipelihara. Dalam praktikum ini terbukti dengan pemisahan App, ActivityList, dan ActivityItem.
- c. State management menggunakan useState Hook memungkinkan aplikasi merespons interaksi pengguna secara real-time tanpa perlu me-reload halaman.
- d. Props adalah mekanisme komunikasi data dari parent ke child component yang menegakkan prinsip unidirectional data flow sehingga aplikasi lebih mudah diprediksi dan di-debug.
- e. Rendering list dengan map() dan key yang unik adalah cara standar menampilkan data array di React secara efisien.
- f. Conditional rendering memungkinkan UI yang adaptif — menampilkan konten berbeda berdasarkan kondisi state, seperti pesan 'Belum ada aktivitas' saat array kosong.
- g. Vite terbukti sebagai build tool modern yang sangat cepat untuk pengembangan aplikasi React, dengan Hot Module Replacement (HMR) yang memperbarui tampilan secara instan saat kode diubah.